Assignment Title: Network Traffic Analysis

Name: Mariia Shevchenko

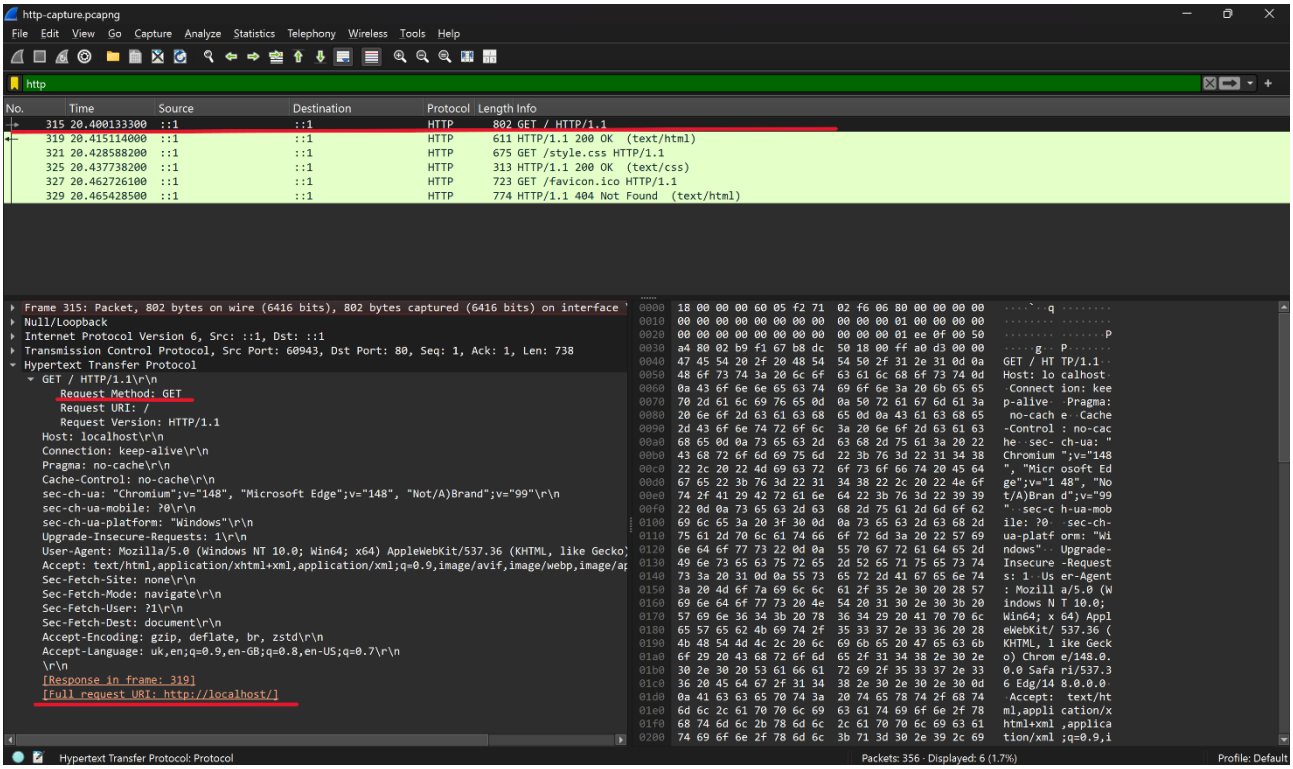
Course Name: Network Security L2

Date: 25.05.2026

Assignment Tasks

Task 1: Comparative Analysis of HTTP and HTTPS Traffic:

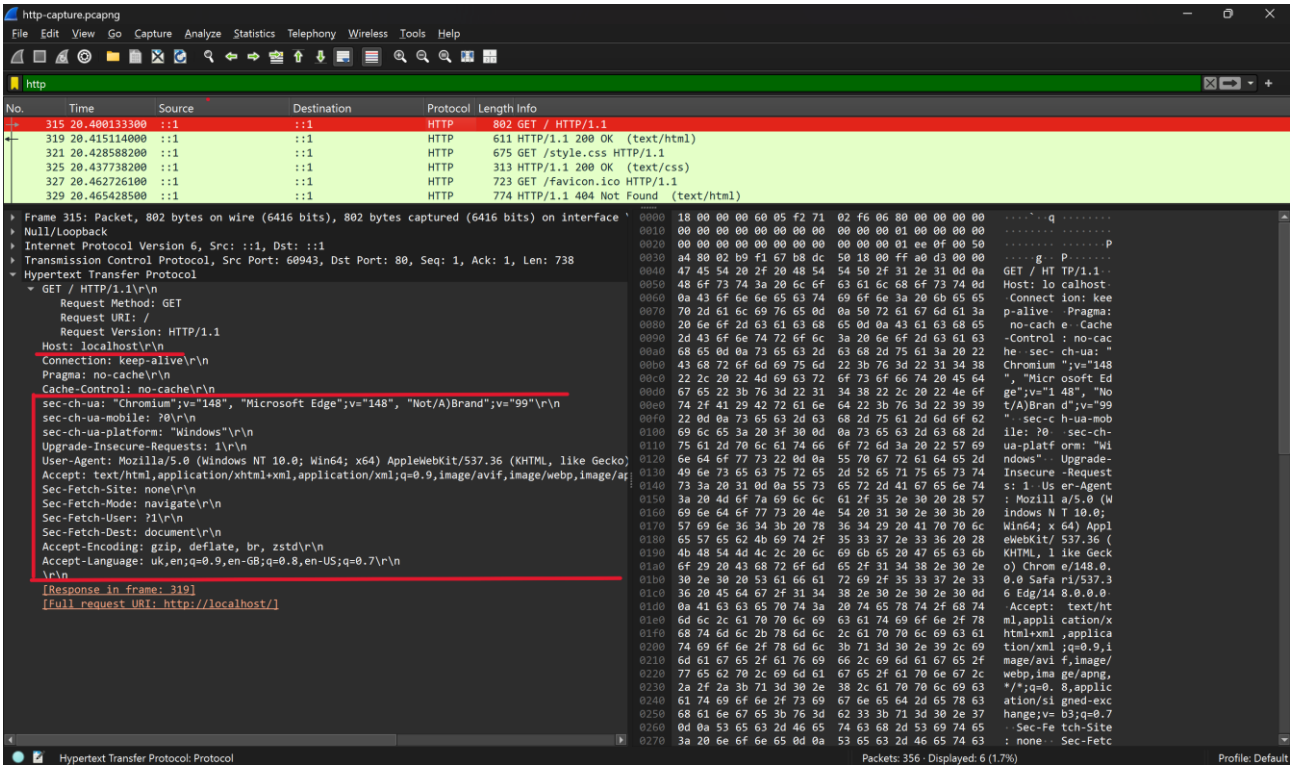
Request method and URL:



- **Request Method:** The browser uses the **GET** method. In web networking, a GET method retrieves information without changing anything on the server.
- **URI:** The request URI is just a forward slash, which stands for the root directory. Wireshark puts the pieces together at the bottom to show the full web address.

Security Implication: Because this is standard, unencrypted HTTP, this entire request is sent in plain text. If an attacker is sitting on the same Wi-Fi network or controls a router along the way, they can easily use a packet sniffer to see exactly what website and what specific pages the user is visiting.

HTTP Headers:

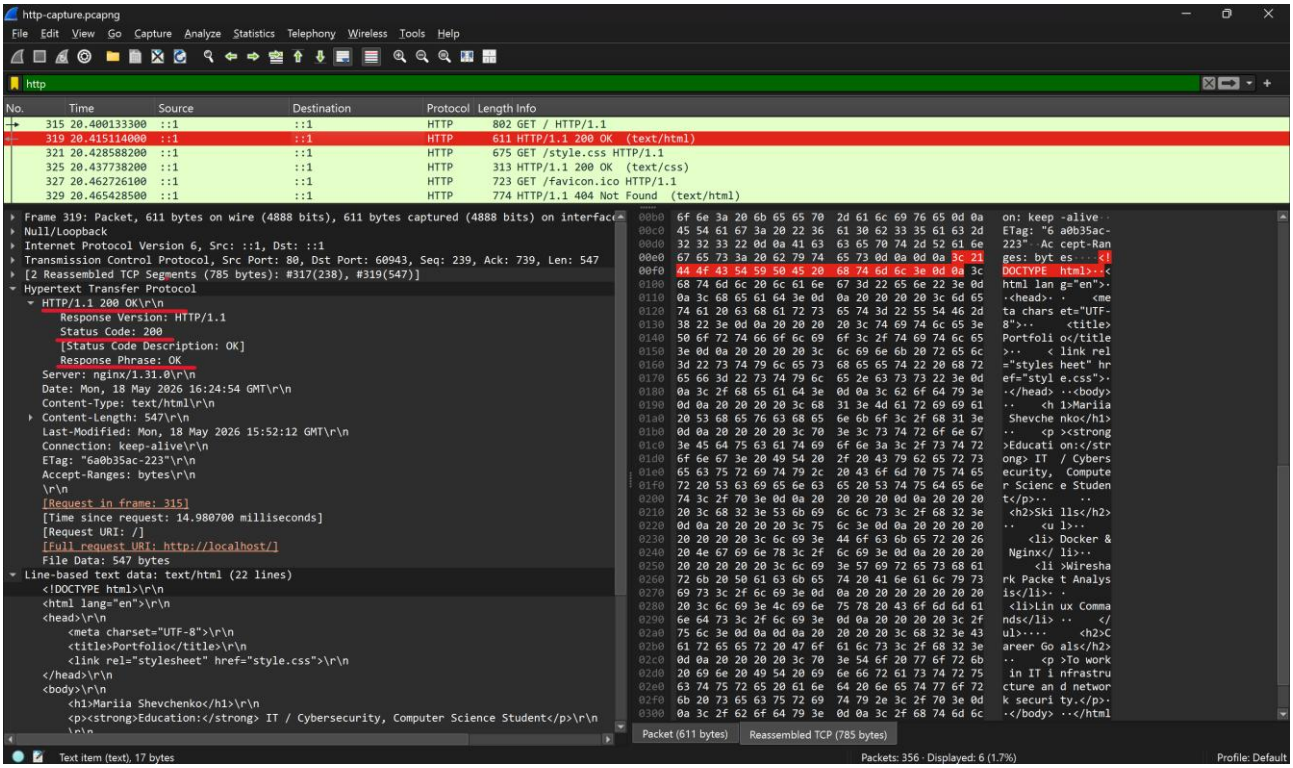


HTTP Headers are additional pieces of information that are sent together with requests and responses. They contain details such as content type, length, HTTP method, and so on, to manage the behavior of HTTP transactions.

- **Host Header:** This line displays Host: localhost. It tells the Nginx server exactly which website domain name the browser is trying to connect to.
- **User-Agent Header:** This long string reveals that the user is running a Windows 10/11 operating system (Windows NT 10.0) and using a Chromium-based browser like Microsoft Edge or Google Chrome.

Security Implication: > When headers are sent in plain text, it creates two major security problems. First, exposing the **Host** header destroys user privacy because an eavesdropper can see exactly what websites a user is visiting. Second, the **User-Agent** header gives away the user's exact operating system and browser version. Attackers love this information because it allows them to target the user with specific software exploits tailored to that exact browser version.

Response status codes:

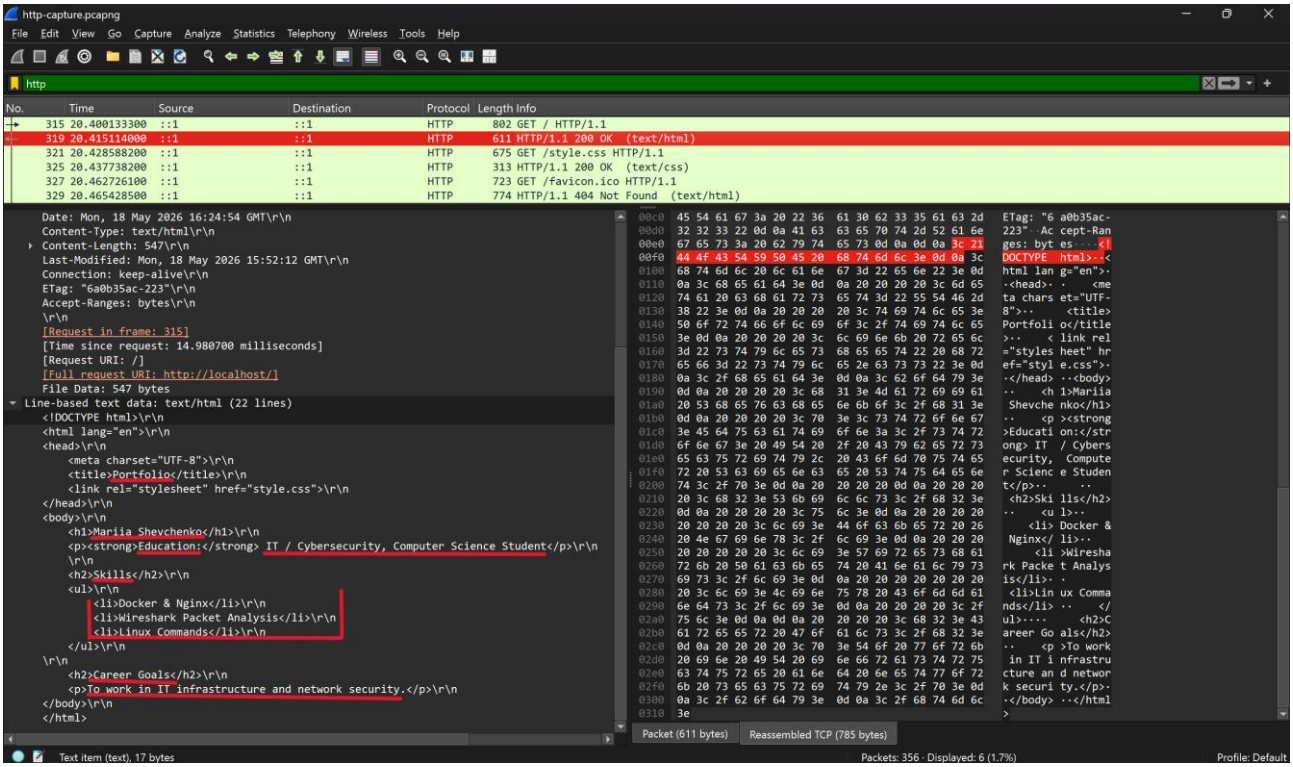


After a client sends an HTTP request, the web server replies with an HTTP response packet that begins with a status code.

- **Status Code 200 OK:** The Nginx server responds with the status code **200 OK**. This code means the request was completely successful. The server found the file the browser asked for and is delivering it back to the user.
- **Content Type Identification:** The packet info also specifies (text/html), letting the browser know that the incoming data stream should be read and rendered as a standard web page.

Security Implication: Because this status code and its response metadata are completely unencrypted, an eavesdropper monitoring the network can see exactly how the server handled the request. They can tell that the resource exists and was successfully retrieved. If a user tries to access a restricted page and receives a 403 Forbidden or a 401 Unauthorized code, an attacker sniffing the traffic instantly learns which directories are protected, helping them map out the web application's security structure.

Readable content in the packet payload:

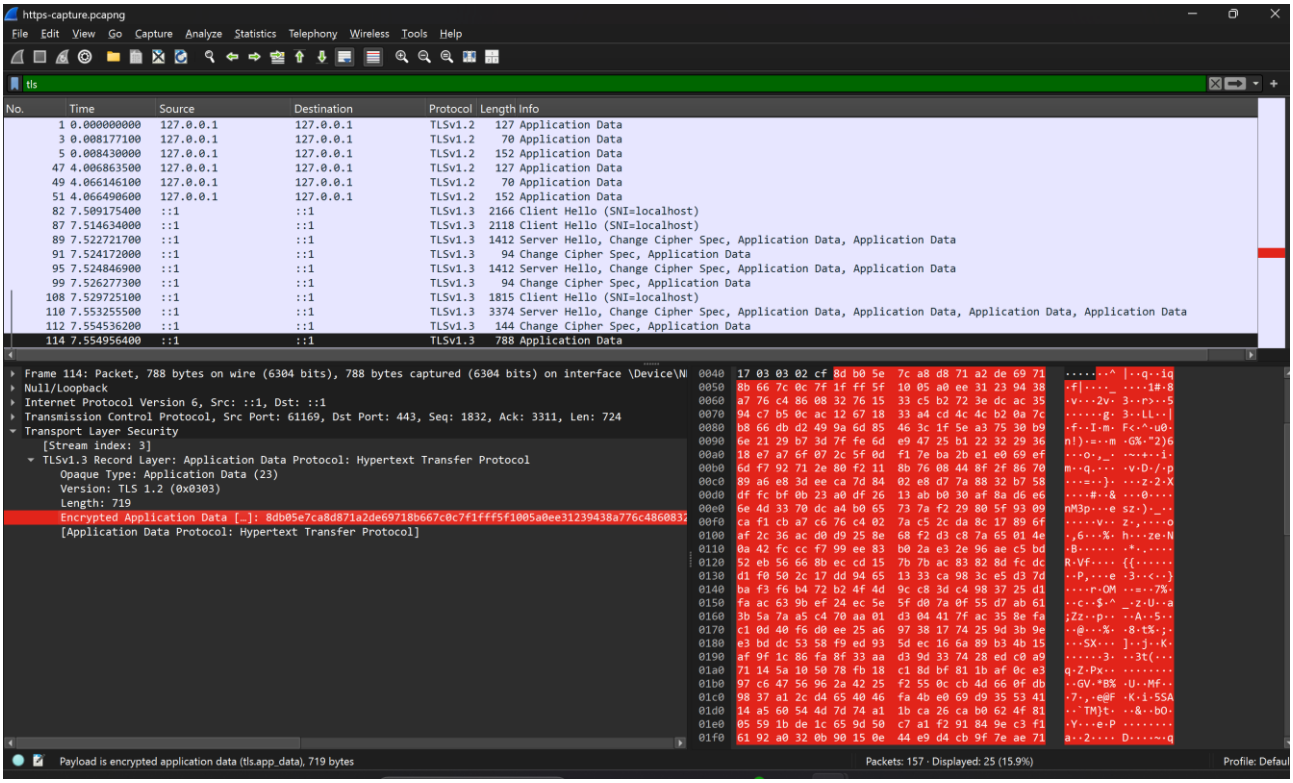


The final layer of our HTTP analysis looks at the actual payload data being sent back to the browser. Because HTTP provides no confidentiality, the actual content of the web page travels over the network exactly as it is written on the server.

- **Readable HTML Data:** Under the **Line-based text data**, we can see the complete, raw HTML source code of the webpage. This is not encrypted or hidden. It clearly reveals the information.

Security Implication: This completely demonstrates the primary vulnerability of HTTP. If a user is filling out a form, checking their bank account, or reading private messages on an HTTP site, a packet sniffer captures that raw data instantly. Attackers do not need complex decoding tools or decryption keys; they simply read the plaintext text directly out of the packet payload pane. This makes data tampering, identity theft, and credential harvesting incredibly easy for bad actors on an unencrypted network path.

The HTTPS Comparison (Encrypted Data):

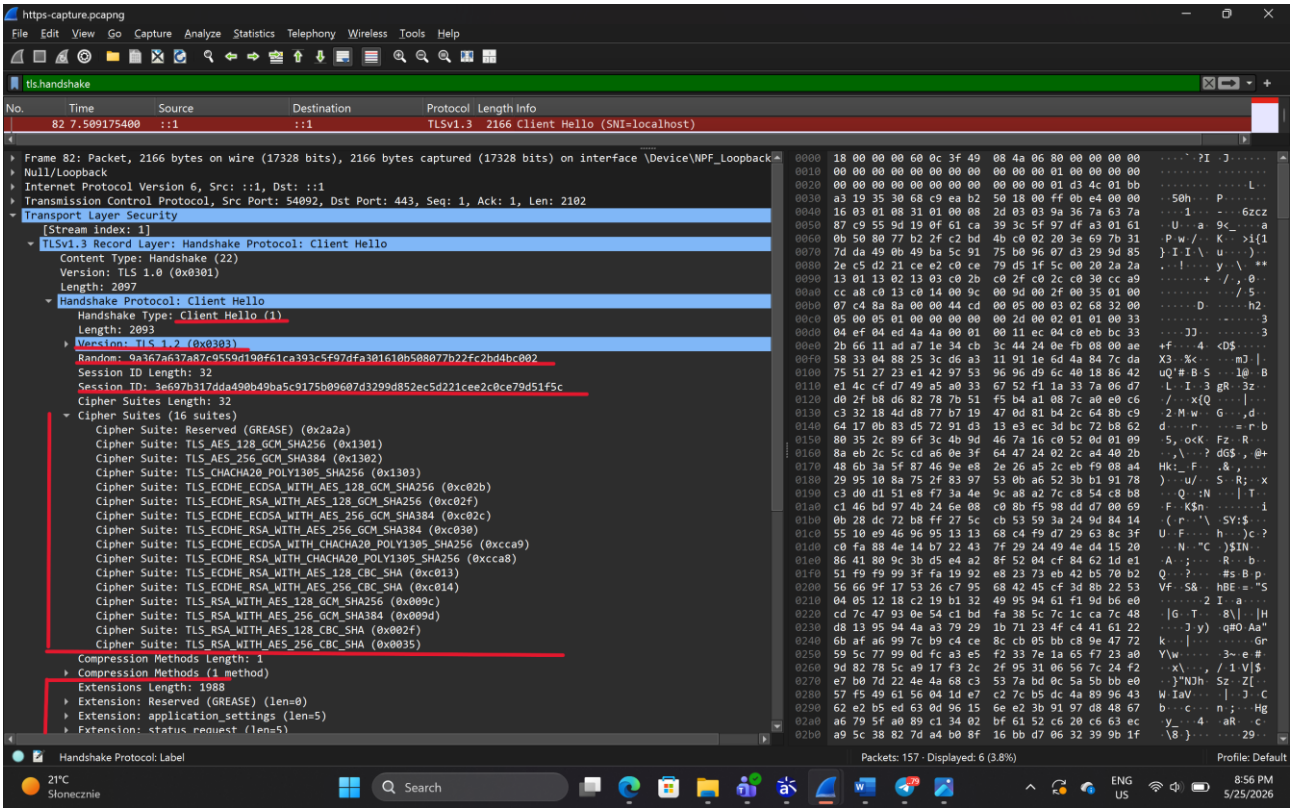


- **Hidden Request Methods, Headers, and Payload:** In our HTTP screenshots, we could easily read the GET method, the Host header, and the HTML code. In this HTTPS packet, all of that information is gone. As shown in the details pane, the entire HTTP layer is wrapped inside **Encrypted Application Data**.
- **Scrambled Hex/ASCII Pane:** There are no readable words or HTML tags. Everything is encrypted.

Security Implication: Because the data is encrypted using symmetric keys created during the TLS handshake, a Man-in-the-Middle attacker sniffing this traffic learns absolutely nothing about the user's data. They cannot see what specific webpage is being viewed, they cannot read any headers, and they cannot steal personal text from the payload.

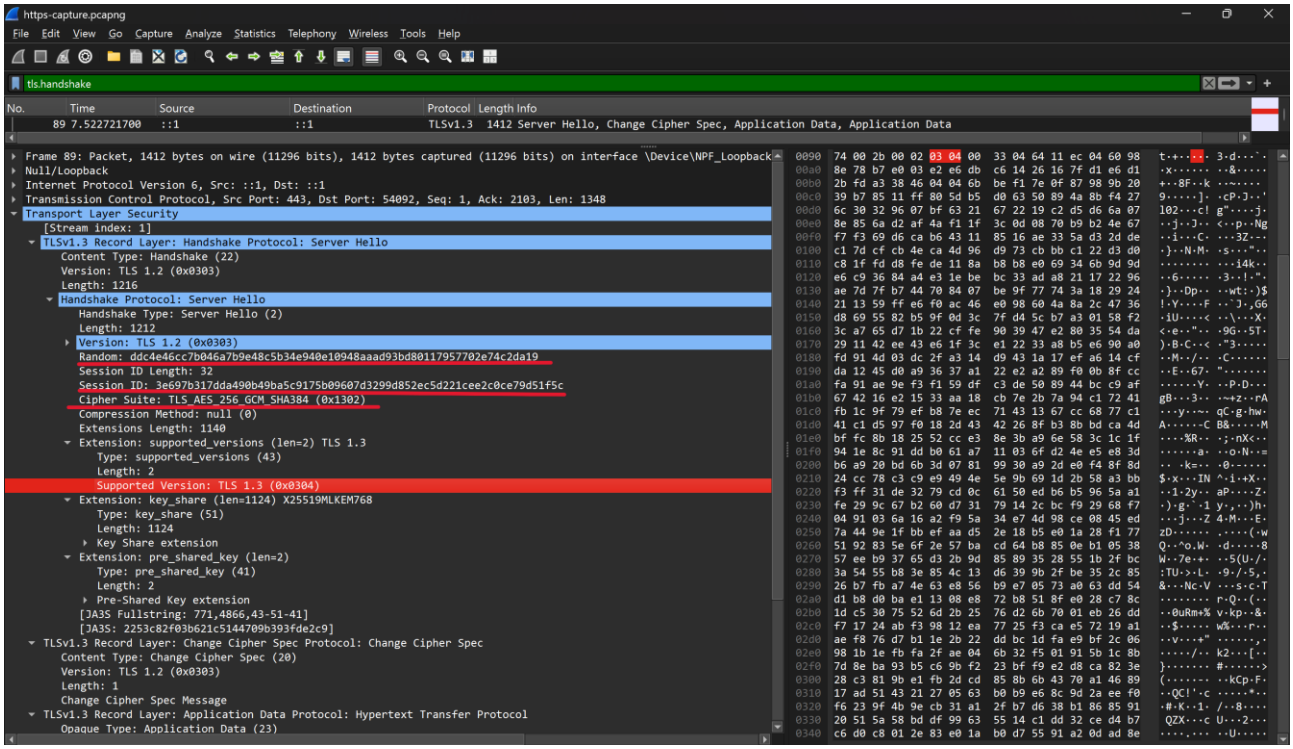
Task 2: TLS Handshake Analysis and Documentation

Phase 1: Client Hello:



- **What Happens:** The client initiates the connection by advertising its technical capabilities. It provides a list of cryptographic algorithms it supports and security extensions it wants to use.
- **Key Technical Details:** The **Version** field identifies the maximum protocol version the client supports. The **Random** field contains a 32-byte random value generated by the browser, which will be used later to generate the master encryption keys. The **Cipher Suites** section lists all the combinations of key exchange, bulk encryption, and hashing algorithms the browser is willing to use.

Phase 2: Server Hello:



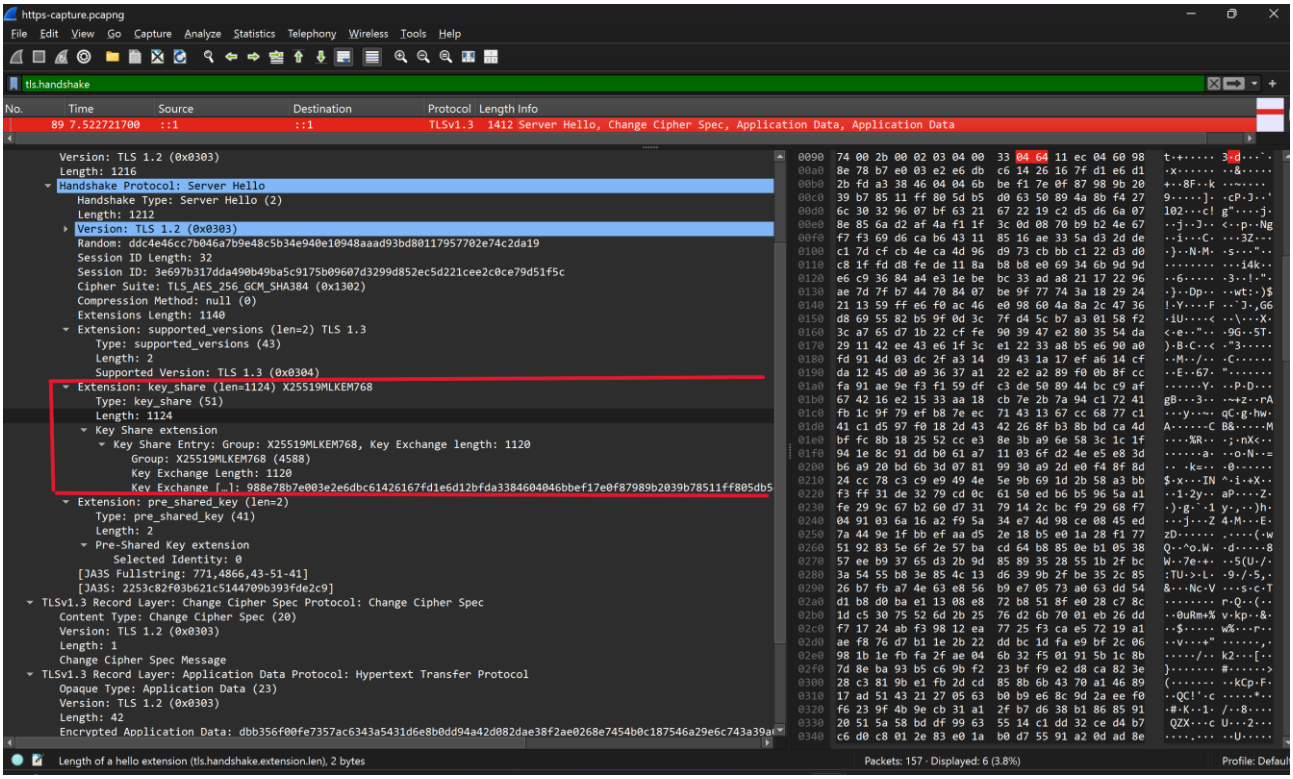
- **Selected Version:** While the record layer states TLS 1.2 for network compatibility, the details pane reveals a supported_versions extension indicating that the server explicitly selected **TLS 1.3 (0x0304)**. This confirms a modern, highly secure connection setup.
- **Chosen Cipher Suite:** The server selected **TLS_AES_256_GCM_SHA384**. This dictates that the session will use AES-256 for authenticated encryption, and SHA-384 for generating message digests.
- **Server Random & Session ID:** The server provides a unique 32-byte **Server Random** value, which both sides will use mathematically to derive the final encryption keys. It also echoes back a **Session ID** which can be used to resume this secure state later without a full handshake quickly.

Security Implication: The Server Hello completes the cryptographic negotiation. By enforcing TLS 1.3 and a strong cipher suite, the Nginx server eliminates vulnerable legacy algorithms that are susceptible to modern network attacks, ensuring robust forward secrecy and data integrity moving forward.

Phase 3 & 4: Certificate and Key Exchange:

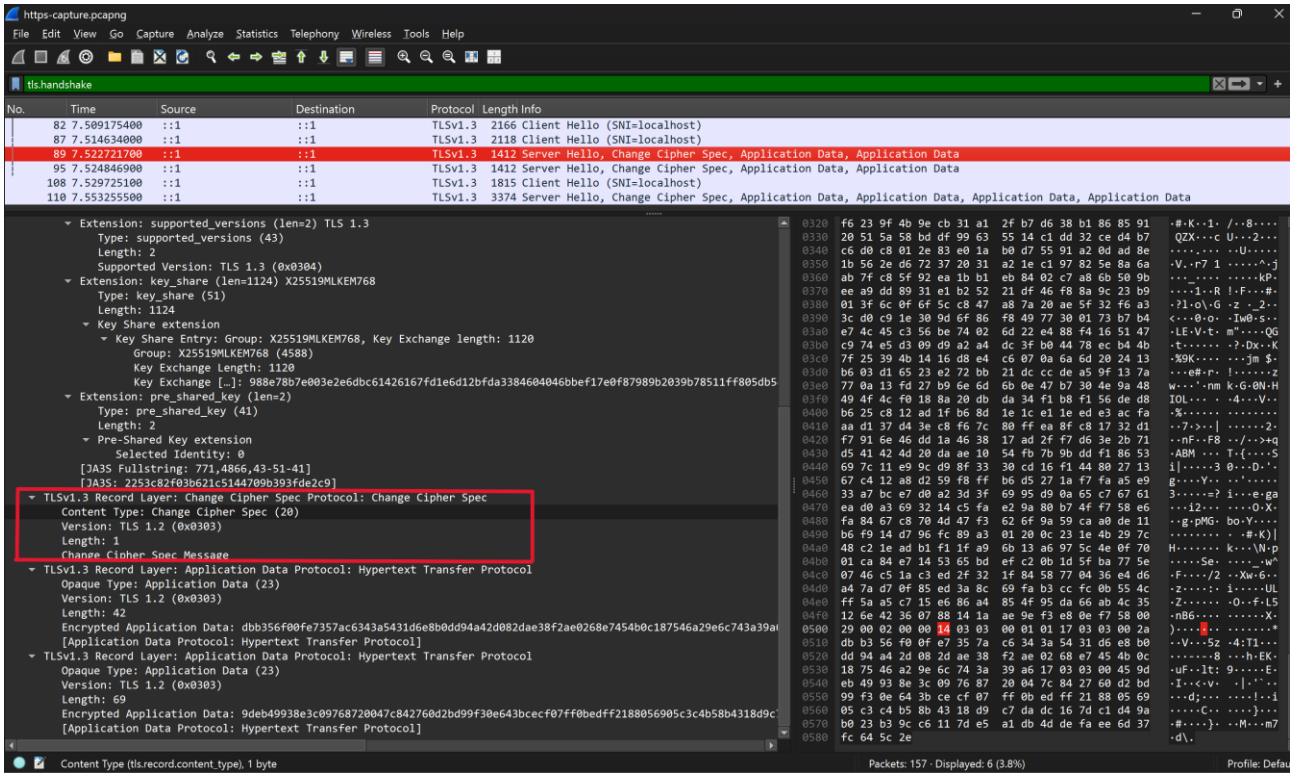
In a modern TLS 1.3 handshake, the Certificate and Server Key Exchange messages do not appear as separate, plaintext packets. Instead, to maximize privacy and prevent eavesdroppers from collecting certificate metadata, TLS 1.3 encrypts these phases.

Phase 5: Server Hello Done / Client Key Exchange:



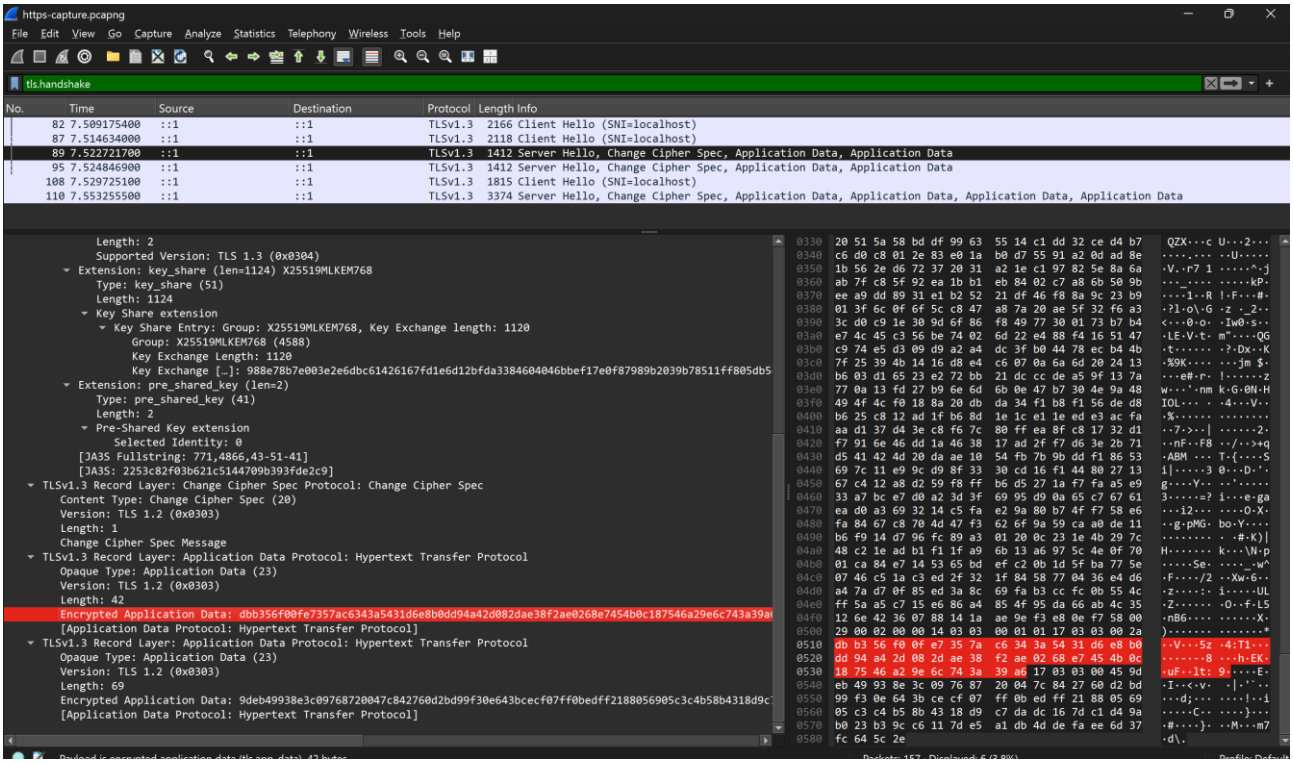
- **What Happens:** During this phase in a TLS 1.3 session, the key parameters are calculated using a process called a Diffie-Hellman key exchange. The client proactively sends its key guesses in the first packet, and the server completes the math immediately upon sending its reply.
- **Key Technical Details:** This process is visible via the **Extension: key_share** block. The capture shows that a hybrid cryptographic algorithm (X25519MLKEM768) is being deployed. This protocol combines elliptic-curve cryptography (X25519) with post-quantum cryptography (ML-KEM) to prevent future supercomputers from breaking the encryption. The server provides its raw public key share bits right here. Once both sides have exchanged these public parameters, they combine them with their private parameters locally to instantly generate the matching symmetric encryption keys.

Phase 6: Change Cipher Spec and Encrypted Handshake Messages:



- **What Happens:** Both the client browser and Nginx server send a Change Cipher Spec flag. Immediately following this flag, they send an encrypted "Finished" message which appears under the encrypted Application Data protocol layers seen directly underneath it.
- **Key Technical Details:** The **Change Cipher Spec** record is clearly parsed by Wireshark out of the multi-message packet block. It indicates a length of exactly `1` byte. This prevents older, rigid corporate firewalls along the network path from dropping the connection when they suddenly encounter unexpected encrypted binary streams. The encrypted Finished records that follow immediately verify that the key generation math was identical on both ends, checking the conversation for data integrity before traffic is cleared to pass.

Phase 7: Application Data:



- **What Happens:** The web browser and Nginx server transition completely into a secure transmission state. Every individual packet exchanged from this point forward is dynamically encrypted using the symmetric keys calculated during the handshake process.
- **Key Technical Details:** When expanding the Transport Layer Security tree in the details pane, the standard application layer fields are replaced entirely by an opaque **Encrypted Application Data** string. Looking at the raw hexadecimal byte pane on the right confirms that any plain text elements, such as the GET method or personal portfolio code discovered during the unencrypted Task 1 HTTP capture, are completely hidden from view.

Task 3: TLS Handshake Diagram Creation: